Compilation (HAI705I)

Master Informatique
Département Informatique
Faculté des Sciences de Montpellier
Université de Montpellier

Examen du 16 janvier 2026

Tous les documents de cours sont autorisés. L'examen dure 2h. Le barème est donné à titre indicatif. Le sujet comporte 3 pages et il y a 4 exercices.

Exercice 1 (5 pts)

On considère la fonction suivante de $\mathbb{N} \times \mathbb{N}$ dans $\mathbb{N}$:

$$myst(n,p) = \begin{cases} 0, \text{ si } n = 0 \\ 0, \text{ si } n > 0 \text{ et } p = 0 \\ myst(m,q) + 1, \text{ si } n = m+1 \text{ et } p = q+1 \end{cases}$$

1. Dérouler au moins 3 exemples d'évaluation de la fonction *myst*.
   Que fait la fonction *myst* ?
2. Écrire la fonction *myst* en PP.
3. Traduire la fonction *myst* en UPP.
4. Traduire la fonction *myst* en RTL.
5. Traduire la fonction *myst* en ERTL.
   A-t-on besoin de registres *callee-save* dans cette traduction ? Justifier pourquoi.

Exercice 2 (5 pts)

Soit le programme PP suivant :

```
w := u + 1;
t := 0;
z := u + v;
t := z;
x := z;
y := z;
t := y;
x := y
```

1. Dessiner le graphe de flot de contrôle du programme.
   Réaliser l'analyse de durée de vie des variables sachant qu'à la fin du programme, les variables x et y sont vivantes.
   Dessiner le graphe d'interférences correspondant.
2. Colorier le graphe d'interférences obtenu à la question précédente avec 2 couleurs en utilisant l'algorithme de Chaitin. Doit-on « spiller » ?
   Même question en utilisant le « coalescing » (on attribue une couleur s'il y en a une de disponible même si le sommet a été « spillé » dans un premier temps).

Exercice 3 (7 pts)

Nous disposons d'une machine virtuelle (VM) à registres, proche de celle du cours mais très simplifiée. L'objectif est de générer un code de cette VM permettant l'interprétation d'automates déterministes, c'est-à-dire la reconnaissance d'un mot par un automate. On suppose que la VM peut gérer des listes LISP, avec des opérations spécialisées :

— `(car R1 R2)` prend la cellule dont l'adresse est dans le registre `R1` et charge son champ `car` dans le registre `R2` ; (`cdr` a le rôle symétrique pour le champ `cdr`) ;

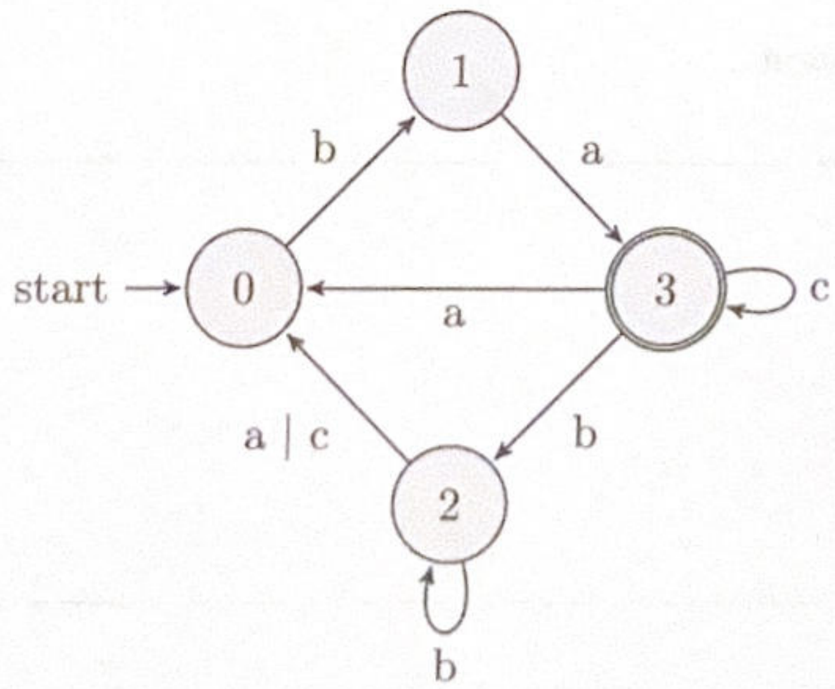
— l'opération de comparaison `(cmp R1)`, utilisée avec un seul opérande, permet d'effectuer des tests de cellules (`consp`, `atom`, `null` en LISP) sur le contenu du registre `R1` en positionnant des drapeaux de manière usuelle ;

— `(bconsp #label)` est une instruction de branchement conditionnel qui effectue le branchement si le drapeau préalablement positionné par `cmp` indique qu'il s'agit bien d'une cellule. Avec `batom` et `bnull`, le branchement est conditionné au fait que la valeur testée est un atome ou `nil`.

Les conventions pour le code d'interprétation des automates sont les suivantes. La donnée (mot à reconnaître) est une liste de caractères (symboles), par exemple `(c b a c)`, contenue dans le registre `R0`. À l'issue de l'exécution, la VM s'arrête et `R0` contient l'état final atteint lors de l'exécution de l'automate, ou `nil`, suivant que le mot a été reconnu ou pas.

Question 1

Soit l'automate ci-dessous, dont l'état initial est 0 et l'ensemble des états finaux est $\{3\}$:



1. Indiquer comment traduire les états de l'automate dans le jeu d'instructions de la VM.

2. Écrire le code VM correspondant à l'automate donné ci-dessus, en le commentant.

On suppose que l'on dispose, en LISP, d'un type de données automate, muni de l'interface fonctionnelle suivante :

- `(auto-etat-liste auto)` retourne la liste des états (entiers) de l'automate : pour celui de l'exemple, (0 1 2 3) ;
- `(auto-init auto)` retourne l'état (entier) initial de l'automate (0 dans l'exemple) ;
- `(auto-final-p auto etat)` retourne vrai si l'état argument est final (dans l'exemple, vrai pour 3, faux pour les autres) ;
- `(auto-trans-list auto etat)` retourne la liste des transitions issues de l'état argument, sous la forme d'une liste.

Question 2

Écrire la fonction LISP `auto2vm` qui prend en argument un automate déterministe (au sens de la structure de données précédente) et retourne le code VM correspondant (c'est-à-dire un code similaire à celui que vous avez écrit dans la question précédente pour l'automate donné en exemple).

1. Spécifier le principe de la génération : comment traduire les états, les transitions, les états finaux, l'état initial, etc.
2. Décomposer le problème en définissant des fonctions annexes pour traiter séparément, chaque transition, chaque état, etc.

Exercice 4 (3 pts)

Les questions qui suivent concernent les éléments du projet compilation : le compilateur, la machine virtuelle, le chargeur.

Question 1

Expliquez comment vous implémentez la machine virtuelle (VM) en termes de structures de données. Pour les informations de la VM, pour lesquelles choisissez-vous de les modéliser comme attributs et lesquelles vous placez dans la mémoire de la VM ? Justifiez vos choix.

Question 2

Expliquez comment s'organise la transformation source à source et la génération de code avec le jeu d'instruction de la VM. Donnez un exemple parlant.

Question 3

Comment chargez-vous la version compilée du chargeur ? À quel moment le faites-vous ? Comment chargez-vous du code avec la version compilée du chargeur ?